BOSTON UNIVERSITY

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 2
Combinational Logic Review

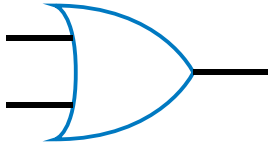
Prof. Douglas Densmore

Announcements

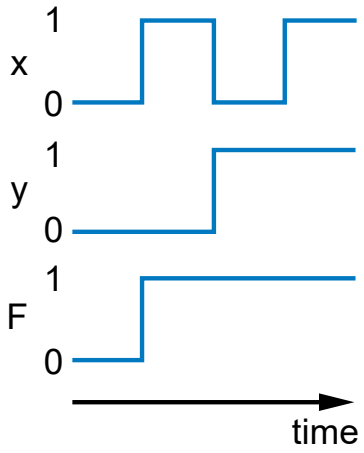
- Don’t forget that the “Syllabus Confirmation” is due Tuesday (9/15) in class.
- Make sure you have access to Blackboard (you need it to get the syllabus confirmation). I don’t want to hear about issues midway through the semester.
- Also make sure to sign up for Piazza - piazza.com/bu/fall2026/engec551
- “Quiz” sheets available on Blackboard. We will do them in class on (9/15). Do them BEFORE class!

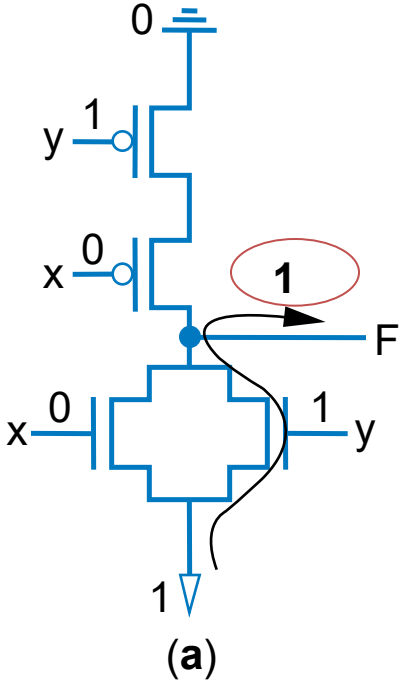
Read Chapter 2 Advanced Digital Design with the Verilog HDL (2nd Edition)

CMOS Gates

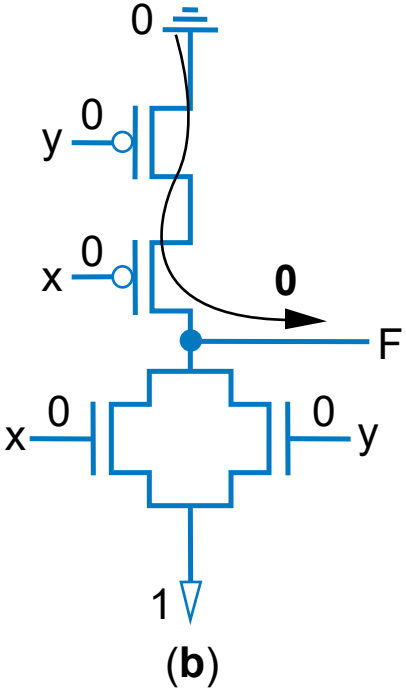


x	y	F
0	0	0
0	1	1
1	0	1
1	1	1



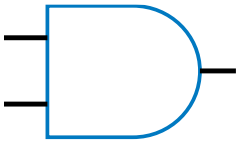


When an input is 1

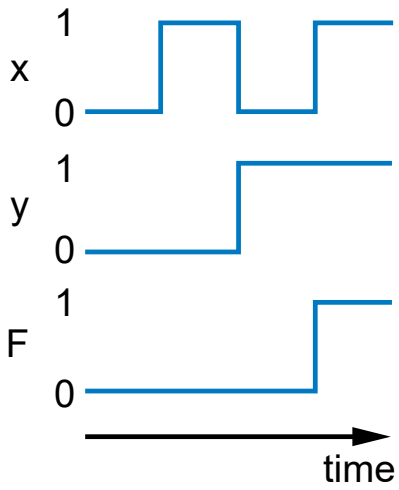


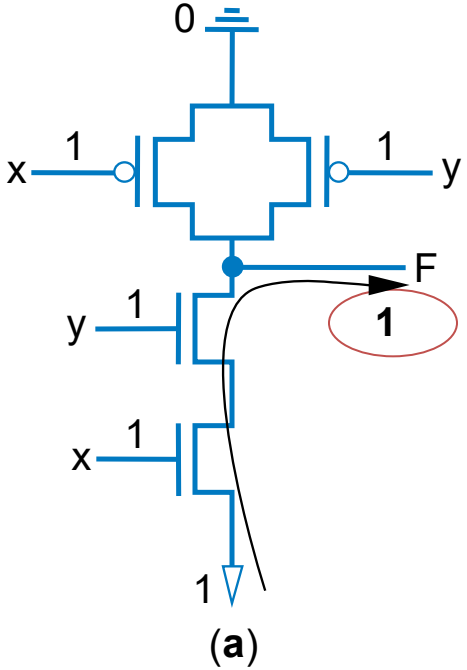
When both inputs are 0

CMOS Gates

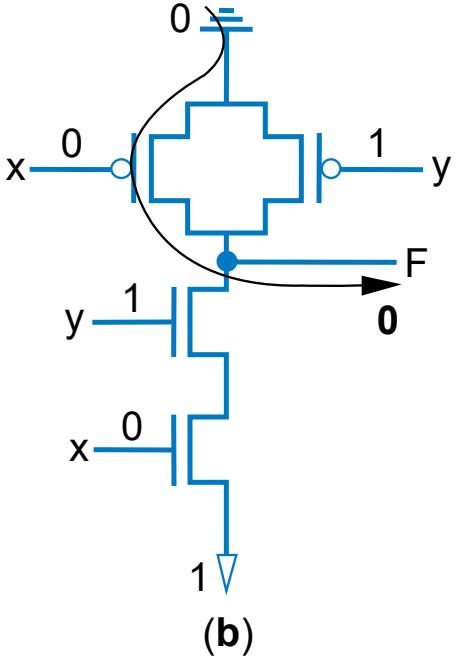


x	y	F
0	0	0
0	1	0
1	0	0
1	1	1





When both inputs are 1



When an input is 0

CMOS Logic

- What was “wrong” with the circuits previously shown from an actual VLSI implementation standpoint?
- How do you “chain” circuits together?
- What are some issues when creating “long chains”? What is this called?

Boolean Algebra Intro

- Developed mid-1800's by George Boole to formalize human thought
  - Ex: "I'll go to lunch if Mary goes OR John goes, AND Sally does not go."
    - Let F represent my going to lunch (1 means I go, 0 I don't go)
    - Likewise, m for Mary going, j for John, and s for Sally
    - Then **F = (m OR j) AND NOT(s)**
  - Nice features
    - Formally evaluate
      - m=1, j=0, s=1 --> F = (1 OR 0) AND NOT(1) = 1 AND 0 = **0**
    - Formally transform
      - F = (m and NOT(s)) OR (j and NOT(s))
        - » Looks different, but same function
        - » We'll show transformation techniques soon
    - Formally prove
      - Prove that if Sally goes to lunch (s=1), then I don't go (F=0)
      - F = (m OR j) AND NOT(1) = (m OR j) AND 0 = 0

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1

a	b	OR
0	0	0
0	1	1
1	0	1
1	1	1

a	NOT
0	1
1	0

Boolean Algebra 1

- Commutative
  - a + b = b + a
  - a * b = b * a
- Distributive
  - a * (b + c) = a * b + a * c
    - Can write as: a(b+c) = ab + ac
  - a + (b * c) = (a + b) * (a + c)
    - (This second one is tricky!)
    - Can write as: a+(bc) = (a+b)(a+c)
- Associative
  - (a + b) + c = a + (b + c)
  - (a * b) * c = a * (b * c)
- Identity
  - 0 + a = a + 0 = a
  - 1 * a = a * 1 = a
- Complement
  - a + a' = 1
  - a * a' = 0
- To prove, just evaluate all possibilities

Example uses of the properties

- Show abc' equivalent to c'ba.
  - Use commutative property:
    - a*b*c' = a*c'*b = c'*a*b = c'*b*a
- Show abc + abc' = ab.
  - Use first distributive property
    - abc + abc' = ab(c+c').
  - Complement property
    - Replace c+c' by 1: ab(c+c') = ab(1).
  - Identity property
    - ab(1) = ab*1 = ab.
- Show x + x'z equivalent to x + z.
  - Second distributive property
    - Replace x+x'z by (x+x')*(x+z).
  - Complement property
    - Replace (x+x') by 1,
  - Identity property
    - replace 1*(x+z) by x+z.

Boolean Algebra 2

- Null elements
  - a + 1 = 1 (Unit property)
  - a * 0 = 0 (Zero property)
- Idempotent Law
  - a + a = a
  - a * a = a
- Involution Law
  - (a')' = a
- DeMorgan's Law
  - (a + b)' = a'b'
  - (ab)' = a' + b'
  - *Very useful!*
- To prove, just evaluate all possibilities

- Absorption
  - a + ab = a
  - a(a+b) =a
- Consensus
  - ab + bc + a'c = ab + a'c
  - (a+b)(b+c)(a'+c) = (a+b)(a'+c)

SOP

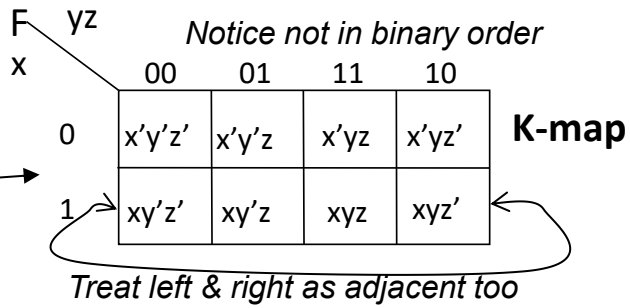
- Truth tables too big for numerous inputs
- Use standard form of equation instead
  - Known as ***canonical form***
  - Regular algebra: group terms of polynomial by power
    - $ax^2 + bx + c$ ($3x^2 + 4x + 2x^2 + 3 + 1 \dashrightarrow 5x^2 + 4x + 4$)
  - Boolean algebra: create sum of minterms
    - ***Minterm***: product term with every function literal appearing exactly once, in true or complemented form
    - Just multiply-out equation until sum of product terms
    - Then expand each term until all terms are minterms

Q: Determine if F(a,b)=ab+a' is equivalent to F(a,b)=a'b'+a'b+ab, by converting first equation to canonical form (second already is)

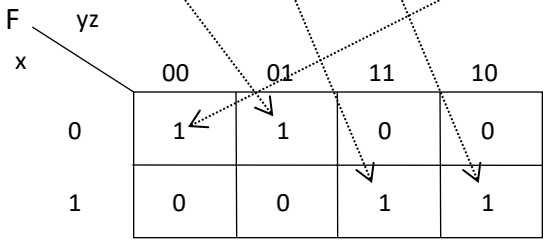
F = ab+a' (already sum of products)
F = ab + a'(b+b') (expanding term)
F = ab + a'b + a'b' (Equivalent – same three terms as other equation)

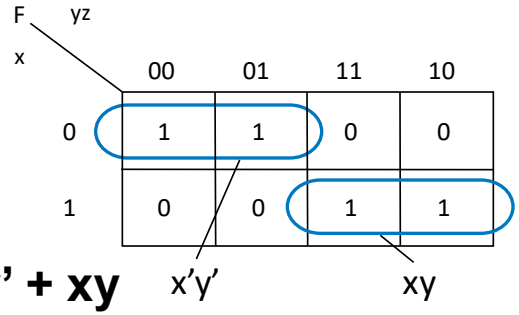
K-Maps 1

- Easy to miss possible opportunities to combine terms when doing algebraically
- ***Karnaugh Maps (K-maps)***
  - **Graphical** method to help us find opportunities to combine terms
  - Minterms differing in one variable are *adjacent* in the map
  - Can clearly see opportunities to combine terms – look for adjacent 1s
    - For F, clearly two opportunities
    - Top left circle is *shorthand* for:
      x'y'z'+x'y'z = x'y'(z'+z) = x'y'(1) = x'y'
    - Draw circle, write term that has all the literals except the one that changes in the circle
      - Circle xy, x=1 & y=1 in both cells of the circle, but z changes (z=1 in one cell, 0 in the other)
    - Minimized function: OR the final terms



F = x'y'z + xyz + xyz' + x'y'z'





F = x'y' + xy

Easier than algebraically:

F = xyz + xyz' + x'y'z' + x'y'z
F = xy(z + z') + x'y'(z + z')
F = xy*1 + x'y'*1
F = xy + x'y'

K-Maps 2

General K-map method

1. *Convert* the function's equation into sum-of-minterms form
2. *Place* 1s in the appropriate K-map cells for each minterm
3. *Cover* all 1s by drawing the fewest largest circles, with every 1 included at least once; write the corresponding term for each circle
4. *OR* all the resulting terms to create the minimized function.

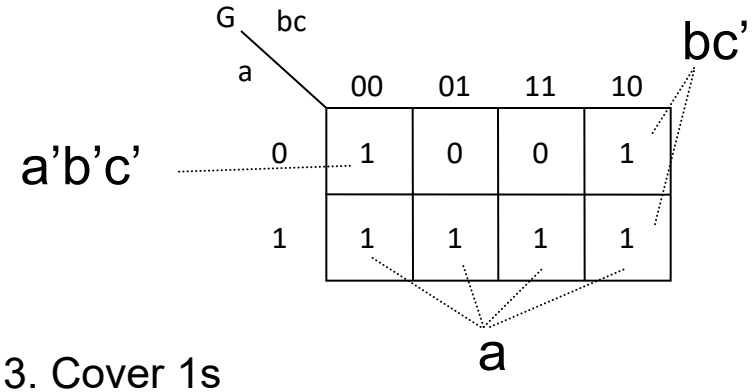
Example: Minimize:

G = a + a'b'c' + b*(c' + bc')

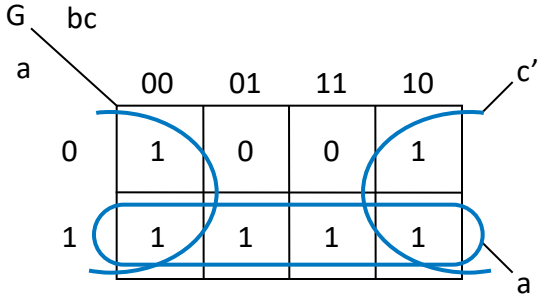
1. Convert to sum-of-products

G = a + a'b'c' + bc' + bc'

2. Place 1s in appropriate cells



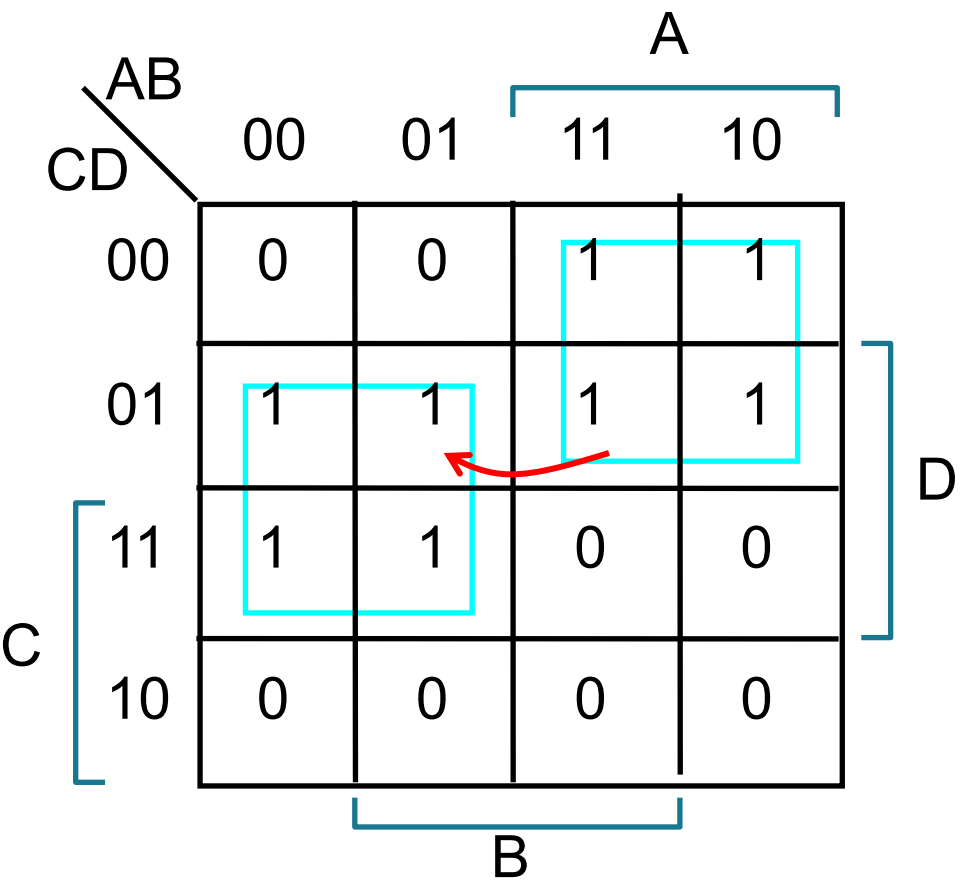
3. Cover 1s



4. OR terms: **G = a + c'**

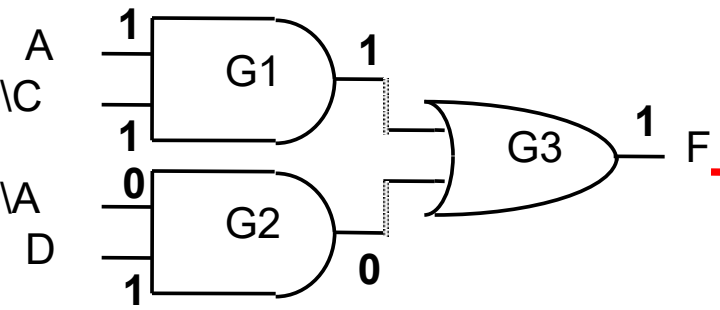
Glitches and Hazards 1

- Glitch: Unwanted switching at the outputs
- Hazard: Component organization which causes glitch
- Occur when different paths through circuit have different propagation delays
- Dangerous if logic causes an action while output is unstable
  - may need to guarantee absence of glitches



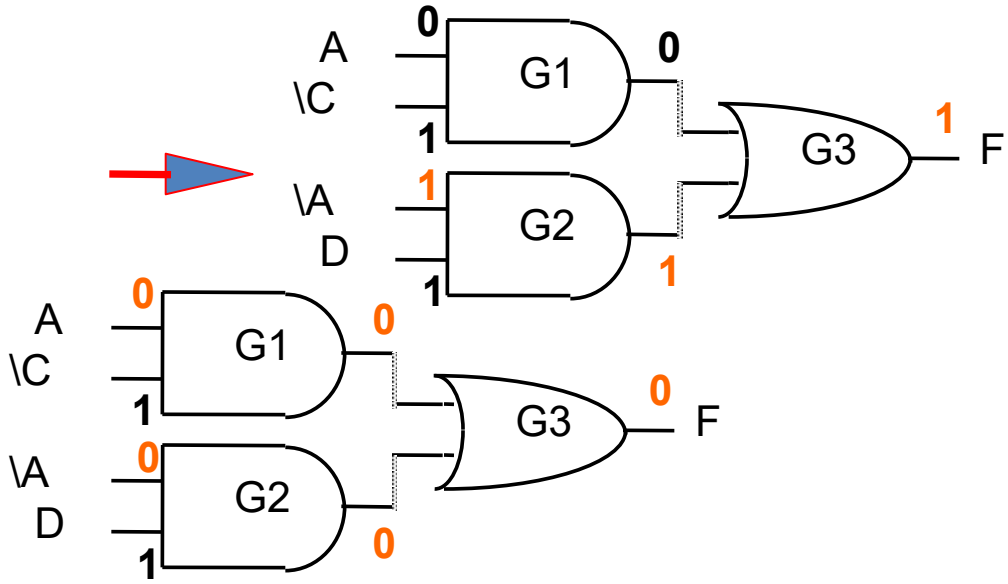
$$F = A\overline{C} + \overline{A}D$$

When the input change by a single bit from 1101 to 0101,
When A goes low A' goes high after an inverter delay and a glitch has happened
The input change spans prime implicants



ABCD = 1101

ABCD = 0101 (A is 1)

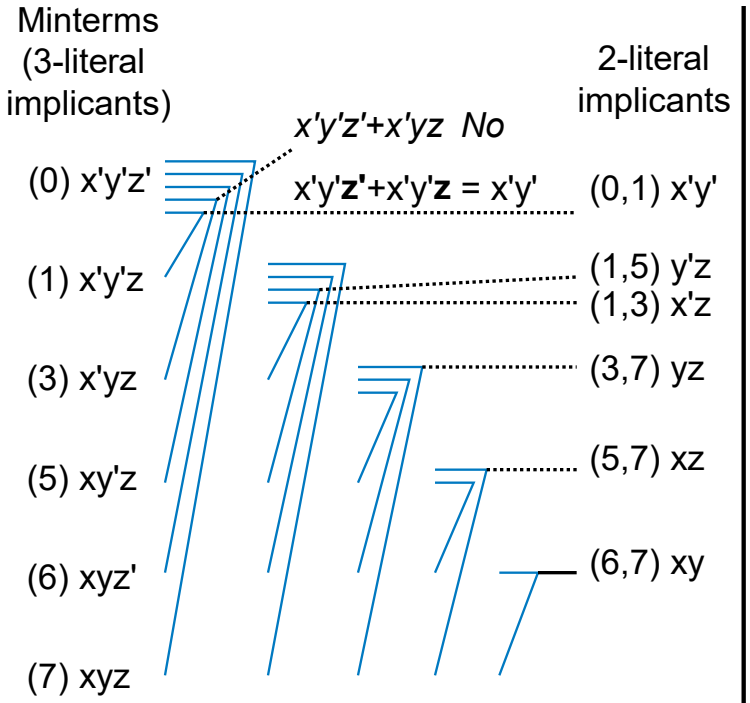


ABCD = 0101 (A is still 0)

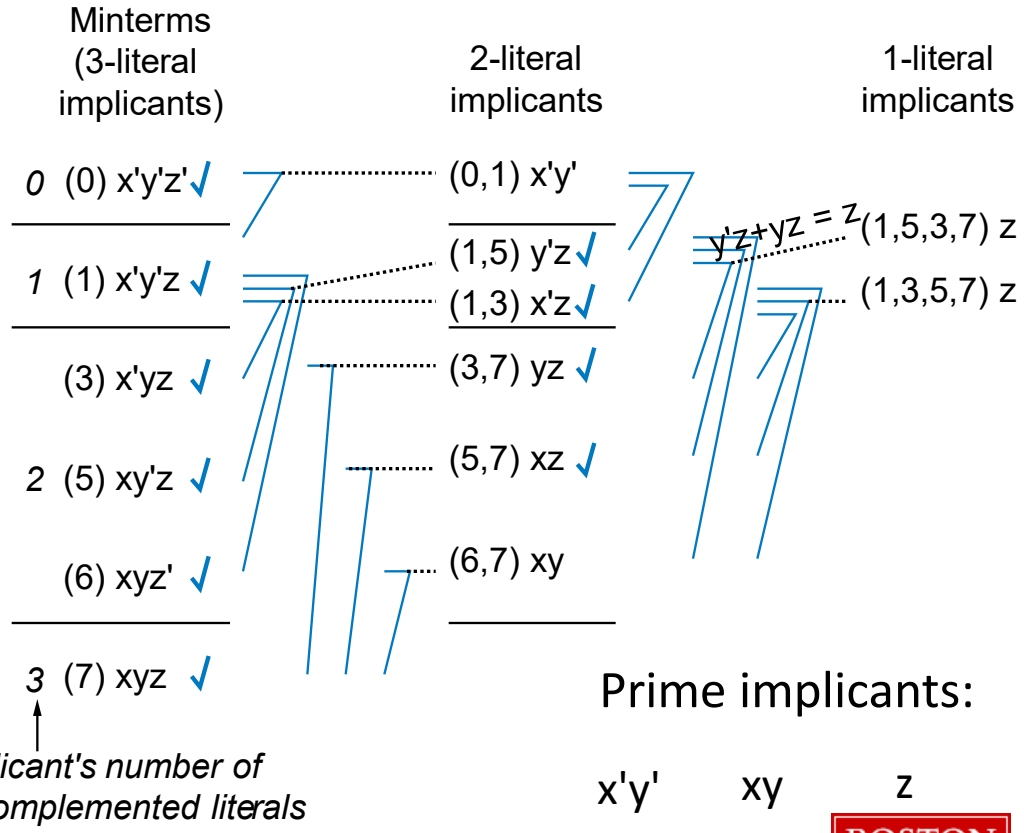
BOSTON UNIVERSITY

Tabular Method Step 1: Determine Prime Implicants

- Example function: F = x'y'z' + x'y'z + x'yz + xy'z + xyz' + xyz



Actually, comparing ALL pairs isn't necessary—just pairs differing in uncomplemented literals by one.

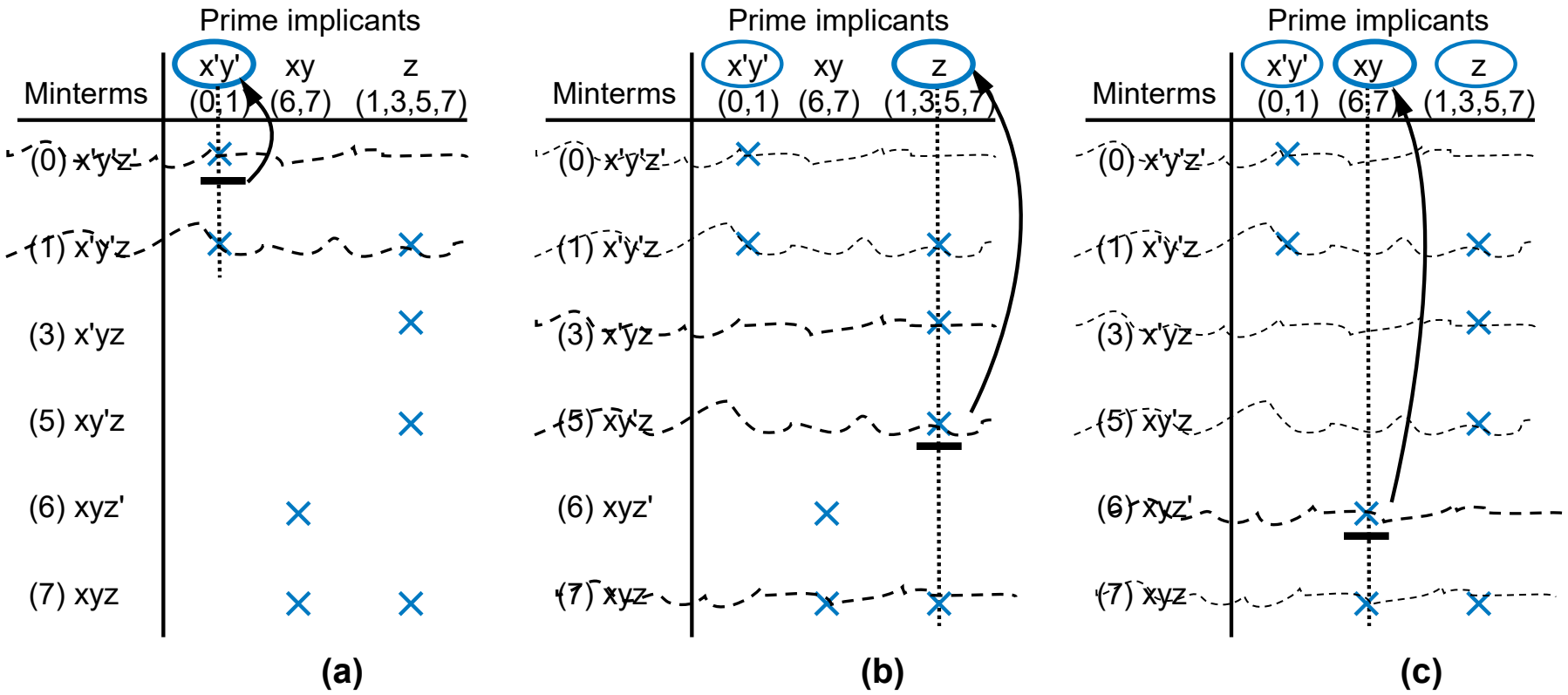


Prime implicants:

x'y' xy z

Tabular Method Step 2: Add Essential PIs to Cover

- Prime implicants (from Step 1): x'y', xy, z



If only one **X** in row, then that PI is essential—it's the only PI that covers that row's minterm.

Next Time

- Sequential Logic Review
- Be sure to get the Combinational Logic Review "Quiz" from Blackboard

Questions?